ListedWheels: Architecture Document

Version 1.0 · Prepared for the ListedWheels team and Avinash · Stack: FastAPI, Pydantic, PostgreSQL, Google Auth

1. Purpose of this document

This document explains **what we are building, how the pieces fit together, and why each decision was made**. It is the reference for anyone who joins later, for future changes, and for finding the cause of an error. The companion document, *ListedWheels: Architecture and Build Guide*, covers the order in which everything is built.

ListedWheels is a trust-focused marketplace for used Cars, Bikes, Cycles and (later) Other vehicles in India. Sellers pay a per-listing fee valid for 12 months. Admin-created promo codes can make a listing free. Buyers contact sellers through WhatsApp.

2. Assumptions to confirm

These were not specified, so this document assumes them. Change them early if they are wrong.

#	Assumption	Why
A1	The frontend is **Next.js (React) with TypeScript and Tailwind CSS**	Listing pages must be indexed by Google (SEO). Server-rendered pages do this best.
A2	Hosting is **AWS** (credits available)	Domain and credits already exist.
A3	Photos live in **S3 behind CloudFront**; videos are **links only** (Drive or WhatsApp)	Keeps storage cost near zero.
A4	Launch city is **Chennai**, language **English**, with Tamil later	Concentrates supply and demand.
A5	Verification is **manual** by admin at launch	API checks cost money per lookup.
A6	Prices are **exclusive of 18% GST**; the platform is GST-registered	Confirmed by the owner.

3. System overview

```
Browser / Phone
      |
      v
Next.js frontend  (pages, UI, SEO)
      |   HTTPS + JSON
      v
FastAPI backend
  +-- Routes        (HTTP endpoints, input and output schemas)
  +-- Services      (business rules)
  +-- Repositories  (SQL queries)
      |
      v
PostgreSQL   S3 + CloudFront (photos)   Razorpay   Google OAuth   SES (email)
```

Design principle: each layer has one job and only talks to the layer directly below it. A route never writes SQL. A repository never decides business rules. This is what makes errors easy to locate.

4. The three backend layers

This is the structure you specified, written as strict rules.

4.1 API routes (the HTTP layer)

- Defines endpoints using the verbs **GET** (read), **POST** (create), **PATCH** (partial update) and **DELETE** (remove).
- Validates incoming data with **Pydantic schemas** and shapes outgoing data with response schemas.
- Checks who is calling (authentication) through a dependency.
- Calls exactly one service function and returns its result.
- **Never** contains business logic or SQL.

4.2 Service layer (the business rules)

- Holds every rule: pricing and GST maths, promo code validation, "one payment equals one vehicle", status transitions, refund rules, enforcement ladder.
- Calls repositories for data and external clients (Razorpay, S3) for outside work.
- Raises **domain errors** (for example `PromoCodeExpired`). The route layer turns them into HTTP responses.
- Is where most unit tests are written, because it is plain Python with no web framework.

4.3 Repositories (the database layer)

- The only place SQL exists. One repository per table group (`users`, `listings`, `payments`, and so on).
- Uses **parameterised queries** only (never string-joined SQL), which prevents SQL injection.
- Returns plain data. It makes no decisions.
- Transactions are opened by the service, so that a payment record and a listing status change succeed or fail together.

Recommended tooling: SQLAlchemy 2.0 Core (or asyncpg) for queries, Alembic for migrations, and `psycopg` or `asyncpg` as the driver.

5. Project folder structure

```
listedwheels/
  backend/
    app/
      main.py                # creates the FastAPI app, registers routers
      core/
        config.py            # settings from environment variables
        security.py          # JWT creation and checking
        errors.py            # domain error classes and handlers
        logging.py
      api/
        deps.py              # current_user, require_admin
        routes/
          auth.py  users.py  listings.py  media.py
          payments.py  promos.py  reviews.py  reports.py  admin.py
      schemas/               # Pydantic models (request and response)
      services/              # business rules
      repositories/          # SQL
      clients/               # razorpay.py  s3.py  email.py  google_auth.py
      db/
        session.py  migrations/
    tests/
      unit/  integration/
    pyproject.toml  Dockerfile
  frontend/
    app/ (Next.js routes)  components/  lib/  styles/  public/
  docs/
```

6. Authentication and authorisation

Login method: Google Sign-In only (no passwords, no SMS OTP).

Flow

1. The frontend shows "Continue with Google" and obtains a Google **ID token**.
2. It sends the token to `POST /auth/google`.
3. The backend verifies the token with Google (signature, audience, expiry), then finds or creates the user.
4. The backend returns a short-lived **access token** (15 minutes) and sets a long-lived **refresh token** in an `HttpOnly`, `Secure`, `SameSite` cookie.
5. Every protected request sends the access token; a dependency decodes it and loads the user.

Roles: `buyer`, `seller`, `dealer`, `admin`. A user can be both buyer and seller. Admin is assigned only in the database, never through the API.

Rules

- Mandatory Terms of Service consent is stored with a timestamp and version before the first listing or contact.
- WhatsApp numbers are shown only to signed-in users, with rate limiting, so scrapers cannot harvest them.

7. Data model (PostgreSQL)

All tables have `id` (UUID), `created_at` and `updated_at`. Money is stored in **paise as integers**, never floats.

Table	Key columns	Notes
`users`	google_sub, email, name, role, status, tos_version, tos_accepted_at	`status`: active, suspended, banned
`seller_profiles`	user_id, type (individual or dealer), shop_name, gst_no, maps_link, whatsapp, verified_at	One per seller
`verification_documents`	user_id, doc_type (RC, AADHAAR, PAN), storage_key, status, reviewed_by, reviewed_at	Private bucket, encrypted

Table	Key columns	Notes
`listings`	seller_id, category, title, make, model, year, price_paise, mileage_km, owners, fuel_type, transmission, city, reg_no_hash, reg_no_masked, description, status, expires_at	Statuses below
`listing_media`	listing_id, kind (photo), storage_key, position	EXIF stripped
`promo_videos`	listing_id, link, method, status, consent_at	Statuses: submitted, approved, needs_changes, rejected, posted
`plans`	category, base_fee_paise, duration_days	Cycles 9900, Bikes 99900, Cars 199900, Other 249900
`orders`	listing_id, user_id, base_paise, discount_paise, gst_paise, total_paise, promo_id, razorpay_order_id, status	One order per listing
`payments`	order_id, razorpay_payment_id, amount_paise, fee_paise, status	Written from webhook
`invoices`	order_id, invoice_no, gstin, place_of_supply, cgst, sgst, igst	Sequential numbers
`credit_notes`	invoice_id, amount_paise, reason	Created on refund
`promo_codes`	code_hash, type, value, category, max_uses, used_count, expires_at, locked_user_id, requires_video, is_active	Soft delete only
`promo_redemptions`	promo_id, user_id, listing_id, redeemed_at	Audit trail
`reviews`	seller_id, buyer_id, rating, text, status	One per buyer per seller
`contact_events`	listing_id, buyer_id, created_at	Gives review eligibility and stats

Table	Key columns	Notes
`reports`	listing_id, reporter_id, reason, status	Feeds the enforcement ladder
`favorites`	user_id, listing_id	
`audit_log`	admin_id, action, entity, entity_id, reason, created_at	Every approve, hide, remove, ban
`bans`	user_id, reg_no_hash, phone_hash, doc_hash, reason	Blocks re-registration

Listing statuses: `draft` → `pending_payment` → `pending_verification` → `live` → (`hidden` | `sold` | `expired` | `rejected` | `removed`).

Key database rules

- A **partial unique index** on `reg_no_hash` where status is `live` or `pending_verification` enforces *one active listing per vehicle*.
- Indexes on `(category, city, status, price_paise)` and `(created_at)` keep search fast.
- Locked fields (category, registration, make, model, year) are changed only through an admin request.

8. API surface

Prefix `/api/v1`. Verbs follow the standard meaning.

Area	Endpoints
Auth	POST /auth/google · POST /auth/refresh · POST /auth/logout · GET /me · DELETE /me (data deletion)
Seller profile	GET, PATCH /me/seller-profile · POST /me/documents
Listings (public)	GET /listings (filters, sort, page) · GET /listings/{id} · GET /listings/{id}/contact
Listings (owner)	POST /listings · PATCH /listings/{id} · POST /listings/{id}/sold · DELETE /listings/{id} (draft only)

Area	Endpoints
Media	POST /listings/{id}/photos/presign · POST /listings/{id}/photos · DELETE /listings/{id}/photos/{photo_id}
Promo video	POST /listings/{id}/video-link
Checkout	POST /orders (computes price and GST) · POST /orders/{id}/promo · POST /orders/{id}/pay
Payments	POST /webhooks/razorpay (signature verified) · GET /me/invoices/{id}
Engagement	POST, DELETE /favorites/{listing_id} · POST /reviews · POST /reports
Admin	GET /admin/queue · POST /admin/listings/{id}/approve, reject, hide, remove · POST /admin/users/{id}/ban · CRUD /admin/promos · GET /admin/promos/{id}/redemptions · PATCH /admin/reviews/{id} · GET /admin/exports/gst

Every list endpoint is paginated. Errors use one shape: `{ "code": "PROMO_EXPIRED", "message": "..." }`.

9. Key flows

9.1 Creating and paying for a listing

1. Seller creates a **draft** with category (locked from now on) and details.
2. Seller uploads photos directly to S3 using pre-signed URLs, so the API server never carries image traffic.
3. Seller uploads RC and ID documents to the private bucket.
4. `POST /orders` calculates price on the server: base fee, minus promo discount, plus 18% GST on the discounted amount. **The client never sends a price.**
5. If the total is greater than zero, Razorpay checkout opens. The result is confirmed by the **webhook**, not by the browser.
6. If the total is zero (a 100% promo), Razorpay is skipped. An order, a zero-value invoice and a redemption record are still created.
7. Listing moves to `pending_verification`. An admin approves it, and `expires_at` is set to 365 days from approval.

9.2 Promo codes

- Codes are random and long (for example `DLR-CHN-8F3K2Q`) and stored hashed.
- Validation checks: active, not expired, uses remaining, category match, user lock.
- Failed attempts are rate limited and return one generic error.
- "Delete" sets `is_active = false`. History stays.

9.3 GST invoice

- Created after every successful payment, including zero-value ones.
- Within the seller's state: CGST 9% plus SGST 9%. Otherwise: IGST 18%.
- Refunds create a **credit note** linked to the invoice.
- A monthly CSV export feeds GSTR-1 and GSTR-3B filing.

9.4 Enforcement

Flag → Hide → Request documents (48 to 72 hours) → Remove → Ban. Every step writes to `audit_log`. A ban writes hashes to `bans`.

9.5 Expiry

A daily scheduled job marks listings `expired` when `expires_at` passes and queues reminders at 30, 7 and 1 days before.

10. Frontend architecture

- **Next.js App Router** with server-rendered listing and search pages for SEO, and client components for interactive parts.
- **Tailwind CSS** driven by design tokens (colour, type, spacing) defined once.
- **Component library** in `components/ui` (Button, Input, Card, Badge, Modal, Toast) and `components/domain` (ListingCard, FilterBar, PriceBreakdown).
- **Data fetching** through a typed API client generated from FastAPI's OpenAPI schema, which keeps frontend and backend types in agreement.
- **Forms** with React Hook Form and Zod, matching the Pydantic rules.
- **Mobile first:** layouts are designed at 360 px width and expanded upward.
- Full visual specification is in the Build Guide.

11. Security

Risk	Control
SQL injection	Repositories use parameterised queries only
Stolen sessions	Short access token, HttpOnly refresh cookie, CORS limited to our domain
Price tampering	Server computes every amount
Forged payment	Razorpay webhook signature check, idempotent handling
Leaked documents	Private S3 bucket, server-side encryption, short-lived signed URLs, admin-only access, deletion after verification
Scraping of numbers	Sign-in required, rate limiting
Location leaks	Strip EXIF from photos; show masked registration
Brute force on promo codes	Rate limit per account and IP, generic error
Admin misuse	Roles, audit log, two-factor on the admin Google account
Secrets	Environment variables or AWS Secrets Manager, never in Git

Legal: Aadhaar and RC copies are sensitive personal data. Follow the DPDP Act, publish a privacy policy and grievance contact, and take legal advice before launch.

12. Deployment on AWS (credit-friendly)

Component	Service	Note
Backend	One small container on App Runner or ECS Fargate, or one EC2 instance	Start minimal
Database	RDS PostgreSQL (smallest instance) or Postgres on the same EC2 for the first months	RDS gives automatic backups
Photos	S3 plus CloudFront	Public bucket for photos, private bucket for documents

Component	Service	Note
Email	SES	Very low cost
Frontend	Vercel free tier, or the same AWS container	
Secrets	AWS Secrets Manager or Parameter Store	
Scheduler	EventBridge or a cron container	Daily expiry job
Monitoring	CloudWatch alarms and a Budgets alert at $5 and $20	Avoids bill surprises

Two environments: **staging** and **production**, with separate databases and Razorpay test and live keys.

13. Quality, errors and observability

- **Logging:** structured JSON logs with a request ID in every line, so one failing request can be traced.
- **Error tracking:** Sentry (free tier) for backend and frontend.
- **Health check:** `GET /health` reports app and database status.
- **Tests:** unit tests on services (pricing, GST, promo, status rules), integration tests on routes against a test database, and a few end-to-end tests of the payment and listing flows.
- **Migrations:** every schema change is an Alembic migration, reviewed before running in production.
- **CI:** on each push, run lint, type checks and tests.

14. Scaling path

1. Launch: one backend container and one small database. This serves several thousand listings.
2. Add caching for search results (Redis) and read replicas only when measurements show a need.
3. Move slow work (emails, image processing) to a background queue.
4. Add Postgres full-text or a search engine only if filters and text search become slow.

15. Decision log

Decision	Reason	Revisit when
Layered backend (routes, services, repositories)	Clear place for every kind of change and error	Never; it scales well
Google login only	No password handling, no SMS cost	Many users lack Google accounts
Money in paise integers	Avoids rounding errors	Never
Video as links	Zero storage cost	Dealers struggle to share
Manual verification	No per-check fees	About 1,000 listings
Server-side pricing	Prevents tampering	Never

16. Open product questions

1. Is "Other vehicles" live at launch, and what does it include?
2. Do dealers pay per listing or get a bulk plan?
3. What is the free-listing cap per promo code, and how long does a free listing last?
4. Is a rejected listing refunded in full? (This document assumes yes.)